ЛАБОРАТОРНАЯ 4 · I2C И МАТРИЧНАЯ КЛАВИАТУРА

Кнопки стенда и I2C простыми словами

Как микроконтроллер узнаёт, какую кнопку нажали, и превращает её в ноту. Теория, 80 вопросов с ответами и примеры для защиты.

Что изменилось после ЛР 3

Музыкальные правила остались: ноты, октавы, длительность и гамма. Теперь ввод идёт от **двенадцати кнопок стенда**. Компьютерный терминал показывает результаты; напечатанные в нём клавиши приложением не используются.

Главная идея работы

Кнопки соединены в строки и столбцы. Микроконтроллер по очереди выбирает строку и узнаёт, какой столбец с ней соединился. Для доступа к этим линиям используется микросхема PCA9538, связанная с STM32 по I2C.

Что можно сказать в начале защиты

«Я перенёс управление музыкальной клавиатурой на кнопки стенда. Матрицу читаю через расширитель PCA9538 по I2C. Программа по очереди выбирает четыре строки и собирает результат для всех 12 кнопок. Одно нажатие подтверждается после 30 мс. В режиме TEST выводится код кнопки, в PIANO она управляет музыкой. Боковая кнопка переключает режимы. Обмен и восстановление после ошибок выполняются короткими шагами, поэтому музыка продолжает работать».

Как учить

Сначала разберись со схемой строк и столбцов. Затем прочитай про I2C, фильтр кнопок и два режима. Вопросы идут от простых к подробностям кода. В конце - задачи и маршрут демонстрации.

Памятка описывает текущий проект lab4, задание 1. По README назначенный номер 6 соответствует этому заданию. Это подготовка по реализации, а не официальный список вопросов преподавателя.

1. Первый запуск и управление

Подключи питание и **DBG USB**. Для звука замкни **SA2.7**. Дополнительные замыкания SA2.1-SA2.6 для встроенной клавиатуры этой программе не требуются.

Терминал: UART-канал FT2232 Interface 1, **57600, 8N1, без flow control**, Local echo выключен. Он служит для вывода сообщений.

Сначала режим TEST

После сброса звук выключен и выбран TEST. Отпусти все кнопки и дай программе подтвердить отпускание. Нажми SB1: должен появиться Key 1. Остальные кнопки дают коды до Key 12. Удержание код не повторяет.

Боковая кнопка PC15 переключает режим

Одно подтверждённое нажатие меняет TEST на PIANO, следующее - обратно. После переключения снова отпусти матрицу перед новой игрой.

Кнопка в PIANO	Действие
SB1-SB7	До, ре, ми, фа, соль, ля, си
SB8 / SB9	Октава выше / ниже
SB10 / SB11	Длительность больше / меньше на 0,1 с
SB12	Семь нот до-си подряд

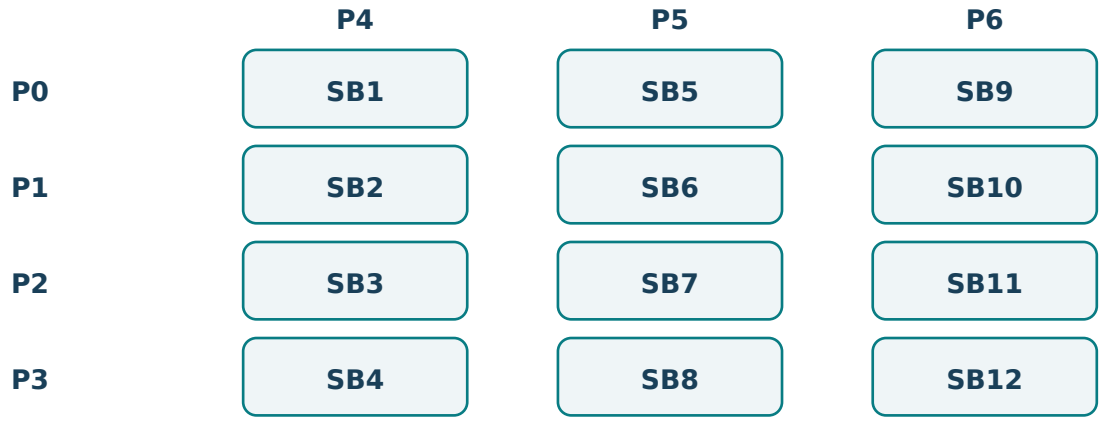
Начальные настройки: первая октава **5/9**, длительность **1,0 с**. Диапазон октав 1-9, времени 0,1-5,0 с. На границе выводится LIMIT, настройка не меняется.

Что проверить первым

В TEST найди физическую SB6 и убедись, что выводится Key 6. Перейди в PIANO, отпусти все кнопки, затем снова нажми SB6. При исходных настройках это Ля 440 Гц.

2. Как подключена матрица

Матрица - это соединение кнопок на пересечениях строк и столбцов. У нас четыре строки и три столбца: 4 × 3 = 12 кнопок.



Строка выбирается программой, столбцы читаются как входы.

Нажатая кнопка соединяет свою строку со своим столбцом. По их сочетанию можно определить номер. Отдельный вывод на каждую из двенадцати кнопок не нужен: используются **4 + 3 = 7 линий**.

Это линии расширителя, а не GPIO STM32

P0-P3 и P4-P6 принадлежат микросхеме **PCA9538**. P0-P3 используются как строки, P4-P6 - как столбцы. P7 этой программе для клавиатуры не нужен.

PCA9538 называется **расширителем ввода-вывода**: он предоставляет восемь дополнительных линий, которыми STM32 управляет по I2C.

Почему номера идут по столбцам

Так кнопки обозначены на схеме. Слева сверху вниз SB1-SB4, посередине SB5-SB8, справа SB9-SB12. Нельзя просто считать ряд слева направо как 1, 2, 3.

Пример

Нулевая строка P0 и столбец P5 означают SB5. Строка P1 и столбец P6 - SB10. В коде строки и столбцы считаются с нуля: `код = row + 4 × column + 1`.

3. Как программа находит нажатую кнопку

Одна строка за раз

Программа выбирает P0: делает эту линию выходом с нулём. Затем читает P4, P5 и P6. Если кнопка этой строки нажата, соответствующий столбец тоже получает ноль.

Дальше так же выбираются P1, P2 и P3. После четырёх строк получается результат полного обхода - **кадр**. Он содержит состояние всех 12 позиций, собранное из четырёх чтений.

В готовой маске единица отмечает нажатую кнопку, ноль - отпущенную. Это уже обработанный результат; в самом регистре Input нажатый столбец, наоборот, читается как ноль.

Почему нажатие даёт ноль

Столбцы подтянуты резисторами к питанию. Пока кнопку не нажали, читается единица. Нажатие соединяет столбец с выбранной строкой, которая держит ноль.

Что с остальными строками

Они остаются **входами**: не навязывают линии ни единицу, ни ноль. Такое состояние называют высокоимпедансным. Оно помогает избежать конфликта выходов при нескольких нажатиях.

Выбранная строка	Что записывается в Configuration
P0	0xFE: только P0 становится выходом
P1	0xFD: только P1 становится выходом
P2	0xFB: только P2 становится выходом
P3	0xF7: только P3 становится выходом

Зачем ждать перед чтением

После выбора строки программа даёт сигналам установиться: ждёт не менее **1 мс**, продолжая выполнять основной цикл. Затем читает столбцы. Это ожидание по времени, без длинной паузы в функции.

Кадр занимает больше четырёх миллисекунд: кроме четырёх выдержек есть обмены I2C и шаги программы. Поэтому отклик кнопки - не строго 30 мс от физического касания.

4. I2C: два сигнала для общения с микросхемой

I2C - способ обмена между микросхемами. По одной общей линии передаются данные, по другой - тактовые импульсы, показывающие, когда читать биты. Нужна также общая земля.

Линия	Назначение	Подключение STM32
SCL	Тактирование обмена	PB8, AF4, I2C1
SDA	Данные в обе стороны	PB9, AF4, I2C1

Кто начинает обмен

STM32 - управляющее устройство: выбирает адрес, начинает обмен и задаёт такты. PCA9538 - адресуемое устройство: принимает настройки или возвращает данные. На той же шине могут быть и другие микросхемы со своими адресами.

Почему open-drain и подтяжки

Линии I2C общие. Устройство может тянуть линию к нулю или **отпустить её**. Когда все отпустили линию, резистор подтяжки создаёт единицу. Такой выход называется open-drain.

Поэтому команда «выдать 1» означает освобождение линии. Она ещё не доказывает, что там действительно высокий уровень: другое устройство может продолжать держать ноль.

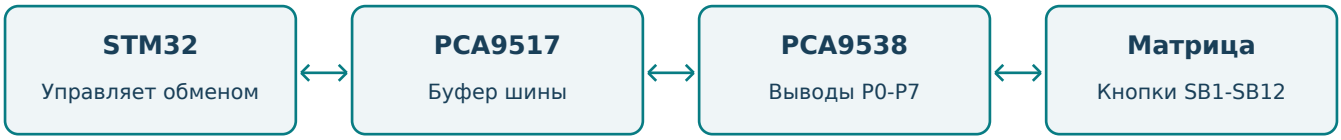
Как передаётся байт

Восемь бит идут от старшего к младшему. После них идёт девятый такт для подтверждения. Обычно данные на SDA меняются при низком SCL и остаются устойчивыми, пока SCL высокий.

START и STOP

START: SDA меняется с 1 на 0 при высоком SCL - начало обмена. **STOP:** SDA меняется с 0 на 1 при высоком SCL - конец обмена. Это специальные исключения из правила устойчивых данных при высоком SCL.

5. По какому пути идут настройки и ответы



STM32 задаёт настройки и считывает результат через I2C.

Две микросхемы с разными задачами

PCA9517 помогает передавать сигналы между частями шины. Здесь это буфер, а не клавиатура. Установка **PD7=1** включает его работу.

PCA9538, DD12 даёт линии матрицы. У него адрес **0x71**. Другой PCA9538 на схеме, DD10 с 0x70, относится к другому узлу.

Имя цепи на схеме может напоминать RESET, но **PD7 в этой схеме включает буфер, а не сбрасывает DD12**. Вывод сброса самого DD12 подключён к высокому уровню.

Адрес 0x71 и байты 0xE2 / 0xE3

0x означает шестнадцатеричную запись числа. 0x71 - семибитный адрес устройства, выбранный его выводами A1=0 и A0=1.

```
запись: (0x71 << 1) | 0 = 0xE2
чтение: (0x71 << 1) | 1 = 0xE3
```

К адресу добавляется один бит направления: 0 для записи, 1 для чтения. Поэтому анализатор может показывать адрес 0x71 или целый адресный байт 0xE2/0xE3 - это два способа представить один обмен.

Что принимает API драйвера

В `i2c_bus_read_reg` и `i2c_bus_write_reg` передаётся именно семибитный адрес **0x71**. Сдвиг и бит направления драйвер добавляет сам.

6. Что хранится в регистрах PCA9538

Регистр - небольшая ячейка управления или состояния. У расширителя есть четыре регистра. Не путай их с регистрами самого контроллера I2C внутри STM32.

Номер и имя	Простой смысл
0 - Input	Прочитать уровни линий
1 - Output	Задать, какой уровень выдавать линиям в выходном режиме
2 - Polarity	Выбрать инверсию при чтении входов
3 - Configuration	Выбрать вход или выход для каждой линии

Главное правило Configuration

1 означает вход, 0 означает выход. Поэтому 0xFF делает все восемь линий входами. Число 0xFE оставляет входами все, кроме P0.

Безопасная начальная настройка

Сначала Configuration=0xFF: все линии отпускаются. Затем Output=0x00: будущие выходы будут выдавать нули. Потом Polarity=0x00: входы читаются без инверсии.

Теперь при выборе строки её можно перевести в выход. В защёлке уже лежит нужный ноль. Остальные линии останутся входами.

Как записывается настройка

```
START -> E2 -> номер регистра -> значение -> STOP
```

После передаваемых байтов устройство должно подтвердить приём. Регистр Configuration имеет номер 3, поэтому для выбора P0 записывается регистр 3 со значением 0xFE.

Почему каждый регистр меняют отдельно

У PCA9538 нет автоматического перехода указателя к следующему регистру. В этой программе для каждого обращения явно отправляют нужный номер регистра.

7. Чтение: зачем repeated START и NACK

Сначала сказать, что читать

Чтобы получить уровни кнопок, STM32 сначала указывает регистр Input с номером 0. Это передача номера регистра, поэтому её адресная часть начинается как запись.

```
START -> E2 -> 0 -> repeated START -> E3
      -> байт входов -> NACK -> STOP
```

Repeated START - повторное начало без STOP между двумя частями. Оно позволяет после выбора регистра сменить направление и прочитать ответ в том же обмене.

ACK и NACK

ACK - получатель подтвердил байт, удержав SDA в нуле на девятом такте. **NACK** - подтверждения нет, линия отпущена.

NACK от устройства при отправке ему адреса здесь означает неудачный обмен. Но **NACK от STM32 после последнего прочитанного байта - нормальное завершение чтения**: больше данных не нужно.

Особый порядок для одного байта в STM32F4

Перед снятием флага ADDR драйвер выключает ACK. Затем читает SR2 и заранее задаёт STOP. Когда появится RXNE, забирает байт из DR. Короткая последовательность защищена от прерываний.

Это порядок команд контроллеру. Он не означает, что на проводе STOP должен оборвать данные посередине: аппаратуре заранее задают правильное завершение однобайтового приёма.

Когда обмен действительно закончен

После одного чтения DR успех ещё не объявляется. Драйвер ждёт, что запрос STOP завершится и флаг BUSY снимется. Только затем выдаёт результат OK.

8. Почему одно нажатие нужно подтверждать

Дребезг бывает и у матричных кнопок

При касании контакты могут несколько раз замкнуться и разойтись. Поэтому новый результат не сразу считается нажатием. Фильтр ждёт **30 мс** устойчивой одиночной кнопки по полным кадрам.

Событие выдаётся **до отпускания**, как только нажатие подтвердилось. Удержание не даёт повтора. Отпускание тоже должно подтвердиться за 30 мс, прежде чем разрешено новое нажатие.

Почему проверяют весь кадр

Одна строка не показывает, что происходит в других. Фильтр получает результат всех четырёх строк и понимает, нажата одна кнопка, несколько или ни одной.

Несколько кнопок

Если одновременно обнаружены две или больше кнопок, ввод блокируется. Если потом оставить одну из них зажатой, события всё равно не будет. Нужно полностью отпустить все и дождаться подтверждения отпускания.

Если быстро перейти с одной кнопки на другую

Без устойчивого полного отпускания такой переход тоже блокируется. Иначе смена или дребезг могли бы стать лишней нотой.

Что такое ghosting

В матрице без отдельных диодов у каждой кнопки три нажатых угла прямоугольника могут создать электрический путь, похожий на четвёртую кнопку. Это называют фантомным нажатием, или ghosting.

Программа не пытается угадать настоящее сочетание. Она отвергает кадр с несколькими кнопками. Например, SB1, SB2 и SB5 могут выглядеть как SB1, SB2, SB5 и SB6 - такой кадр не должен запустить ноты.

9. Что происходит при смене TEST / PIANO

Боковая кнопка и матрица - разные входы

PC15 читается напрямую как GPIO, а SB1-SB12 - через PCA9538 по I2C. Нажатие PC15 даёт ноль; программа переводит его в логическое «нажата» и проверяет дребезг 30 мс.

Удержание PC15 переключает режим только один раз. Зажатая при включении боковая кнопка не считается новым нажатием: сначала её нужно отпустить.

При подтверждённом переключении

Действие программы	Зачем
Меняет TEST на PIANO или обратно	Выбирает смысл матричных кнопок
Останавливает ноту или гамму	Звук завершается при переходе режима
Сохраняет октаву и длительность	Выбранные настройки не теряются
Заново разрешает матрицу только после отпускания	Зажатая кнопка не превращается в действие нового режима
Ставит сообщение в очередь UART	В терминале видно новый режим

Пример

В TEST держим SB6 и нажимаем PC15. Режим станет PIANO, но Ля автоматически не начнётся. Нужно отпустить всю матрицу, подтвердить отпускание и снова нажать SB6.

Если в одном проходе совпали два события

Переключение режима имеет приоритет. Матричное нажатие в этом проходе не используется для ноты. Это убирает случайное действие на границе режимов.

10. Как обмен работает без зависания программы

Вместо ожидания - короткие шаги

Драйвер I2C хранит, какой этап обмена выполняется сейчас. Вызов service проверяет флаг, при готовности делает следующий шаг и возвращается. Если ещё рано, тоже возвращается.

Это **неблокирующий автомат**: основной цикл продолжает отсчитывать ноты, читать PC15 и отправлять сообщения. I2C здесь обслуживается опросом регистров, без I2C-прерываний и DMA.

Как узнать результат

Результат	Что означает
IDLE	Нет запущенного обмена
PENDING	Обмен ещё выполняется
OK	Обмен успешно закончен
ERROR	Обмен завершился неудачно

Если функция запуска вернула true, это только означает, что запрос принят. Результат проверяется позднее. Два обмена одновременно этот драйвер не запускает.

Тайм-аут и восстановление

Обычный обмен ограничен 10 мс. При неудаче программа сначала пытается закончить STOP. Если шина не освобождается, выполняет восстановление. Восстановление тоже ограничено: 30 мс от его начала.

Неудачный кадр матрицы отбрасывается. Повтор настройки расширителя начинается после паузы 50 мс от завершения неудачного обмена. Ошибка не считается доказательством, что все кнопки отпущены.

Когда клавиатуры нет

Отсутствующий расширитель обнаруживается во время обменов. Само приложение продолжает работу, сообщает о проблеме и пробует снова. PC15 и окончание уже начатой ноты должны продолжать обслуживаться.

11. Как освобождают застрявшую шину

Почему одного сброса I2C1 недостаточно

I2C1 находится внутри STM32, а PCA9538 - снаружи. Сброс первого не перезапускает второе. Внешняя микросхема может продолжать удерживать SDA в нуле и ждать оставшуюся часть обмена.

Что делает драйвер

Сначала до 2 мс даёт завершиться STOP и забирает оставшийся принятый байт, если он есть. Если этого мало, выключает управление I2C, временно переводит SCL/SDA в обычные open-drain GPIO и делает до девяти импульсов SCL.

Эти такты помогают внешнему устройству закончить оставшийся байт и подтверждение. Затем программа формирует STOP, возвращает AF4 и заново настраивает I2C1.

Почему это тоже не длинная пауза

Импульсы формируются отдельными шагами с интервалами 1 мс. Между шагами основной цикл занимается остальными задачами. Общий предел восстановления - 30 мс.

Если SCL кто-то держит в нуле

«Отпустить SCL» не означает, что линия поднялась. Драйвер проверяет её реальный уровень. Пока она низкая, импульс не засчитывается. После предела времени попытка завершается, а программа продолжает работу.

Что программой исправить нельзя

Постоянное электрическое замыкание линии может сохраняться. Восстановление помогает после прерванного обмена, но не гарантирует устранение аппаратной неисправности.

Особенность BERR

В документации ES0206 описано ложное появление BERR у этого семейства STM32. Если BERR одиночный, без других ошибок, код учитывает и очищает его, продолжая обмен. Другие ошибки и тайм-аут обрабатываются отдельно.

12. Почему I2C работает примерно на 80 кГц

Для I2C берётся частота шины APB1

Ядро работает на 120 МГц. APB1 после деления на четыре - **30 МГц**. Именно её частота используется при настройке I2C1; удвоение таймеров к I2C не относится.

```
CCR = ceil(30 000 000 / (2 * 80 000)) = 188
f_SCL = 30 000 000 / (2 * 188)
      = примерно 79 787 Гц, или 79,8 кГц
TRISE = 30 + 1 = 31
```

Ceil означает округление вверх. Так расчётная частота не выходит за выбранные 80 кГц. CCR задаёт время тактов SCL, а TRISE связан с допустимым нарастанием сигнала в Standard-mode.

Зачем выбран такой предел

В ES0206 описано ограничение repeated START при Standard-mode 88-100 кГц и определённых условиях фронтов или удержания такта устройством. Выбранные 80 кГц находятся ниже этого диапазона.

Какие флаги проверяет драйвер STM32

Флаг	Простой смысл
SB	Начальное условие START сформировано
ADDR	Адресная часть прошла, нужно завершить её обработку
TXE / BTF	Можно записывать данные / байтовая передача завершена
RXNE	Есть принятый байт
BUSY	Шина занята

Регистры DR, SR1/SR2, CR1/CR2 принадлежат I2C1 внутри STM32. Input и Configuration принадлежат внешнему PCA9538. Это два уровня управления.

Расчётная частота ещё не измерение. Реальные фронты, подтяжки и точность HSI проверяют на стенде.

13. Музыка, сообщения и основные файлы

Звук унаследован из ЛР 3

TIM2 с частотой 60 МГц формирует PWM на PA5/AF1. Для Ля 440 Гц: PSC=0, ARR=136363, CCR1=68182. SysTick с шагом 1 мс помогает отсчитать длительность и переходы гаммы.

Новая нота или SB12 заменяет текущую игру. Смена октавы и длительности влияет на следующий запуск. Уже начатая нота и вся гамма сохраняют исходные настройки.

Почему UART не должен задерживать музыку

Сначала событие сохраняется в очереди из **16 отчётов**. В отчёте уже записаны его нота, частота и настройки. Поздняя отправка не заменит их значениями другой ноты.

TX UART вмещает **512 байт**. Если очередь отчётов полна, новые отчёты отбрасываются и растёт `app.dropped_reports`. Опрос и музыка продолжаются: потеря сообщения не отменяет выполненное действие кнопки.

Кто за что отвечает

Файл	Задача
i2c_bus.c	Обмен регистрами и восстановление I2C
keypad.c	Обход строк, полный кадр и фильтр нажатий
button.c	Проверка боковой кнопки PC15
app.c	TEST/PIANO, действия кнопок, очередь отчётов
piano.c / sound.c	Музыкальные правила / настройки TIM2
clock.c / serial.c	Часы / очереди UART

В каждом проходе main вызывается `app_update`. Он обслуживает UART, PC15, музыку, клавиатуру, звук и отправку одного отчёта. Ввод UART читается небольшими порциями и отбрасывается; он не становится музыкальной командой.

Вопросы 1-10. Как пользоваться программой

Короткие ответы для устной защиты.

1. Что нового в ЛР 4?

Музыкальная клавиатура теперь управляется кнопками стенда. Для их чтения добавлены I2C, расширитель PCA9538 и сканер матрицы.

2. Откуда берутся команды?

Из физических кнопок SB1-SB12. UART служит для сообщений, а набранный в терминале текст приложением игнорируется.

3. В каком режиме программа запускается?

В TEST, без звука. Кнопки дают сообщения Key 1-Key 12, чтобы проверить распознавание.

4. Как перейти к игре?

Нажать боковую кнопку PC15, затем отпустить матрицу перед новым нажатием. Следующее нажатие PC15 вернёт TEST.

5. Что делает кнопка SB8 в TEST?

Выводит только Key 8. Октаву она меняет уже в PIANO. Смысл матричной кнопки зависит от режима.

6. Что делают SB1-SB7 в PIANO?

Запускают до, ре, ми, фа, соль, ля, си выбранной октавы. SB6 при исходных настройках - Ля 440 Гц.

7. Что делают SB8 и SB9 в PIANO?

SB8 выбирает октаву выше, SB9 ниже. Номера октав ограничены диапазоном 1-9.

8. Что делают SB10 и SB11?

Увеличивают и уменьшают длительность на 0,1 с. Разрешён диапазон 0,1-5,0 с. На границе появляется LIMIT.

9. Для чего SB12?

Запускает семь нот до-си подряд с выбранной октавой и длительностью каждой ноты.

10. Нужен ли терминал для ввода нот?

Нет. Он показывает коды, ноты, настройки и ошибки. Даже символ 6 из терминала не заменяет нажатие SB6.

Вопросы 11-20. Матрица и сканирование

11. Что такое матричная клавиатура?

Кнопки на пересечениях строк и столбцов. Нажатие соединяет одну строку с одним столбцом.

12. Сколько линий нужно для 12 кнопок этой матрицы?

Семь: четыре строки и три столбца. Их предоставляет расширитель, а STM32 общается с ним по I2C.

13. Что такое PCA9538?

Микросхема расширения ввода-вывода с восемью линиями. Их режимы и уровни можно задавать и читать через I2C.

14. Какие линии расширителя используются?

P0-P3 - четыре строки, P4-P6 - три столбца. P7 в этой программе не используется для клавиатуры.

15. Как определяется нажатая кнопка?

Одну строку переводят в выход с нулём и читают столбцы. Столбец с нулём показывает кнопку выбранной строки.

16. Почему остальные строки не выдают единицу?

Они оставлены входами, чтобы не создавать конфликт двух выходов при нескольких нажатых кнопках.

17. Почему нажатие читается нулём?

Столбец подтянут к питанию. Нажатая кнопка соединяет его с выбранной строкой, которая держит ноль.

18. Что такое полный кадр?

Результат обхода всех четырёх строк: 12-битная маска, где единица отмечает нажатую кнопку. Фильтр получает её целиком, а не одну строку.

19. Как идут физические номера SB?

По столбцам сверху вниз: слева 1-4, посередине 5-8, справа 9-12. Это соответствует обозначениям схемы.

20. Как из строки и столбца получить код?

Строка и столбец считаются с нуля: код = row + 4 × column + 1. Для P1 и P6 это 1 + 4 × 2 + 1 = 10, то есть SB10.

Вопросы 21-30. I2C и подключение

21. Для чего SCL и SDA?

SCL задаёт такты обмена, SDA передаёт данные в обе стороны. Для соединения устройств нужна и общая земля.

22. Какие выводы STM32 подключены к I2C?

PB8 - SCL, PB9 - SDA. Используются I2C1 и альтернативная функция AF4.

23. Почему на I2C нужен open-drain?

Несколько устройств делят линии. Каждое может тянуть линию вниз или отпускать её; высокий уровень создают резисторы подтяжки.

24. Команда выдать 1 гарантирует единицу на линии?

Нет. В open-drain она только отпускает линию. Другое устройство ещё может держать её в нуле.

25. Кто управляет обменом в этой работе?

STM32 начинает обмен, выбирает адрес и задаёт такты. PCA9538 принимает настройки или возвращает данные.

26. Какой адрес у расширителя клавиатуры?

Семибитный 0x71. На плате его адресные выводы A1=0 и A0=1.

27. Почему на проводе адрес выглядит как E2 или E3?

К семи битам адреса добавляется направление. 0x71 сдвигается на один бит: запись даёт E2, чтение E3.

28. Можно ли вместо 0x71 передать 0x70?

Для этой клавиатуры нельзя: DD12 имеет 0x71. DD10 с 0x70 на стенде относится к другому узлу.

29. Что делает PD7?

Единица включает буфер шины PCA9517. Это не вывод сброса расширителя клавиатуры DD12.

30. Используется ли INT расширителя для каждой кнопки?

Нет. В этой реализации матрица регулярно сканируется через обмены I2C. Сигнал INT для запуска нажатий не требуется.

Вопросы 31-40. Регистры и обмены

31. Что означает бит в Configuration?

Единица выбирает вход, ноль - выход. Поэтому FF делает все восемь линий входами.

32. Зачем Output заранее равен нулю?

При переводе выбранной строки в выход на ней сразу появляется нужный ноль. Уровень уже подготовлен до включения выхода.

33. Зачем Polarity=0?

Чтобы входные уровни читались без инверсии. Нажатый столбец остаётся нулём и так распознаётся программой.

34. Какой регистр читают для кнопок?

Input с номером 0. Из результата берутся биты P4-P6, то есть три столбца.

35. Почему сначала Configuration=FF, а затем Output=00?

Сначала освобождают все линии, затем меняют будущие выходные уровни. Так не создаётся лишнее воздействие на матрицу во время начальной настройки.

36. Какие значения выбирают строки?

FE выбирает P0, FD - P1, FB - P2, F7 - P3. В каждом случае только один бит Configuration равен нулю.

37. Что такое START и STOP?

Начало и конец обмена I2C. START - спад SDA при высоком SCL, STOP - подъём SDA при высоком SCL.

38. NACK всегда означает ошибку?

Нет. NACK на адресе здесь означает неудачу. Но NACK от STM32 после последнего прочитанного байта нормально сообщает, что больше читать не нужно.

39. Зачем repeated START при чтении регистра?

Сначала отправляется номер регистра, затем направление меняется на чтение без STOP между частями обмена.

40. Как принимают один байт на STM32F4?

Выключают ACK до снятия ADDR, читают SR2, задают STOP. При RXNE читают DR. Короткий порядок настройки защищён от прерываний; OK ждёт завершения STOP и BUSY.

Вопросы 41-50. Нажатия без повторов

41. Как подавляется дребезг матрицы?

Одна и та же одиночная кнопка должна подтверждаться полными кадрами 30 мс. Только после этого появляется событие.

42. Нота запускается ровно через 30 мс после касания?

Не обязательно. Кроме 30 мс подтверждения нужно время обхода строк и I2C. Фильтр видит результат только после целого кадра.

43. Удержание повторяет код или ноту?

Нет. Одно подтверждённое нажатие выдаёт одно событие. Это отличается от возможного автоповтора компьютерной клавиатуры в ЛР 3.

44. Зачем подтверждать отпускание?

Чтобы краткий отскок контакта не разрешил ещё одно нажатие. Все кнопки должны быть отпущены по наблюдаемым кадрам не менее 30 мс.

45. Что будет с кнопкой, зажатой при включении?

Она не станет новым нажатием. Фильтр сначала требует подтверждённого полного отпускания.

46. Что будет при двух одновременно нажатых кнопках?

Ввод блокируется. Программа не выбирает одну из них и не пытается играть аккорд.

47. Если после двух кнопок отпустить одну, оставшаяся сработает?

Нет. Блокировка сохраняется до устойчивого отпускания всех кнопок. Затем нужно новое одиночное нажатие.

48. Можно ли сменить одну кнопку другой без полного отпускания?

Такой переход блокируется, если прежнее отпускание ещё не подтверждено. Это защищает от неоднозначной последовательности.

49. Что такое ghosting?

Несколько замкнутых кнопок создают путь, похожий на ещё одну нажатую кнопку. Например, три угла прямоугольника могут выглядеть как четыре. Многокнопочный кадр отвергается.

50. Почему после смены режима нужно отпустить матрицу?

Чтобы удерживаемая или ещё не подтверждённая кнопка не превратилась в действие нового режима. Фильтр запускается заново с требования полного отпускания.

Вопросы 51-60. Приложение и сообщения

51. Что происходит со звуком и настройками при смене режима?

Текущая нота или гамма останавливается. Выбранные октава и длительность сохраняются.

52. Почему PC15 не переключает режим много раз при удержании?

У неё тоже есть проверка дребезга 30 мс и одно событие на переход к нажатию. Новое событие требует подтверждённого отпускания.

53. Если PC15 и матрица дали событие в одном проходе?

Приоритет имеет смена режима. Матричное событие в этом проходе не используется для музыкального действия.

54. Что будет, если во время гаммы нажать SB3?

Начнётся отдельная ми с выбранными сейчас настройками. Новая нота заменяет текущую гамму.

55. Смена октавы во время гаммы меняет оставшиеся ноты?

Нет. Вся начатая гамма хранит исходные настройки. Новая октава и длительность используются при следующем запуске.

56. Чем очередь отчётов отличается от TX UART?

В очереди до 16 событий с параметрами. Затем они превращаются в текст для TX на 512 байт. Это два разных места хранения.

57. Зачем сохранять параметры прямо в отчёте?

Чтобы поздний вывод описал событие в момент его появления. Иначе к отправке текущая нота могла бы уже смениться.

58. Что будет, если очередь отчётов заполнится?

Новые отчёты отбрасываются, растёт app.dropped_reports. Сканирование и музыка продолжаются, действие кнопки не отменяется из-за потери сообщения.

59. Почему ввод терминала не переполняет обработку бесконечным циклом?

Программа за проход отбрасывает не больше 16 принятых UART-байтов, затем возвращается к остальным делам. Эти байты не передаются piano как команды.

60. Кто за что отвечает в коде?

I2c_bus выполняет обмен, keypad собирает и фильтрует матрицу, app выбирает TEST/PIANO, piano управляет нотами, sound настраивает TIM2.

Вопросы 61-70. Ошибки и восстановление

61. I2C здесь работает через прерывания?

Нет. Он обслуживается короткими проверками регистров в основном цикле, без I2C-прерываний и DMA. UART и SysTick при этом используют свои прерывания.

62. True из функции запуска означает успешный обмен?

Нет. Это означает, что запрос принят и стал PENDING. Позже проверяют OK или ERROR. Один драйвер ведёт один обмен за раз.

63. Какой тайм-аут у транзакции?

10 мс. Он не даёт бесконечно ждать готовности. Дальше запускается ограниченная по времени процедура завершения и восстановления.

64. Что делается после сбоя?

Сначала до 2 мс ждут STOP. Если шина не освободилась, выполняют GPIO-восстановление. Общий предел восстановления от его начала - 30 мс.

65. Почему делают девять импульсов SCL?

Чтобы дать внешнему устройству такты для оставшегося байта и подтверждения, затем сформировать STOP. Импульс засчитывается после фактического подъёма SCL. Если она зажата, счёт не продвигается.

66. Почему reset I2C1 не всегда освобождает SDA?

Он сбрасывает контроллер внутри STM32, но не внешнюю PCA9538. Её состояние или постоянное внешнее удержание линии могут сохраниться.

67. Что будет при постоянно зажатой SCL?

По завершении 30 мс попытка восстановления закончится. Программа не останется ждать в бесконечном цикле, но само электрическое замыкание программой может не устраниться.

68. Что происходит с кадром и следующей попыткой после ошибки?

Кадр отбрасывается. После завершения неудачного обмена ждут 50 мс до повтора настройки. Фильтр требует полного отпускания, поэтому ошибка не создаёт ложное нажатие.

69. Любой BERR сразу обрывает передачу?

Нет. С учётом ES0206 одиночный BERR без других ошибок очищается и учитывается, а обмен продолжается. Другие ошибки и тайм-аут обрабатываются отдельно.

70. Какие значения задают примерно 80 кГц?

При APB1=30 МГц выбран CCR=188. Получается около 79787 Гц. В Standard-mode TRISE=30+1=31.

Вопросы 71-80. Расчёты и защита

71. Почему выбрали 80 кГц, а не 100 кГц?

Учли ES0206: при определённых условиях Standard-mode 88-100 кГц может нарушать время repeated START. 80 кГц находится ниже этого диапазона.

72. Нужно ли удваивать APB1 при расчёте I2C?

Нет. I2C1 настраивается от 30 МГц. Удвоение, дающее 60 МГц, здесь относится к таймеру TIM2.

73. Откуда берётся звук?

TIM2_CH1 создаёт PWM на PA5/AF1. Сигнал через SA2.7 идёт к усилителю и BA1. Для таймера используется 60 МГц.

74. Какие настройки TIM2 у Ля 440 Гц?

PSC=0, ARR=136363, CCR1=68182. Частота равна 60000000 / (ARR+1), а высокий участок занимает половину периода.

75. Кто отсчитывает длительность ноты?

SysTick даёт миллисекунды, а piano сравнивает время с началом ноты. TIM2 создаёт быстрые импульсы, но не отсчитывает сам музыкальную секунду в этом приложении.

76. Поддерживается ли переход счётчика миллисекунд через ноль?

Да. Используется беззнаковая разность now - since. Для этих коротких интервалов при регулярном обслуживании она корректна и через переполнение.

77. Какие основные флаги I2C проверяются?

SB - начало, ADDR - адрес, TXE - готовность записи, BTF - завершение байта, RXNE - принятый байт, BUSY - занятая шина.

78. Что проверили программные тесты?

Фильтр всех 12 кодов, матрицу и ghosting, I2C и восстановление, два режима, музыку, UART и запуск прошивки. Код выполняется в модели Cortex-M4; реальные фронты и звук требуют стенда.

79. Ответы есть, а звука нет. Что смотреть?

Сначала выбран ли PIANO, нажата ли SB6 после отпускания, замкнут ли SA2.7. Затем - PA5/AF1, CEN, ARR, CCR1 и lab4_fault. TEST сам по себе не запускает ноты.

80. Что показать логическим анализатором?

SCL около 79,8 кГц, адрес 0x71 или байты E2/E3, запись Configuration для строк и repeated START при чтении. На PA5 отдельно можно измерить около 440 Гц для Ля.

Практика 1. Разбор кнопок и блокировки

Ситуация	Что должно произойти
В TEST нажать и держать SB4	Один Key 4 после подтверждения, до отпускания
Держать SB4 и перейти в PIANO	Звука от удержания нет. Нужны отпускание всех и новое нажатие.
Нажать SB1 и SB2 одновременно	Ввод блокируется, новые ноты не запускаются
После двух кнопок оставить SB1 зажатой	Она всё ещё не считается новым нажатием
Полностью отпустить и затем нажать SB1	После подтверждений отпускания и нажатия ввод снова работает
Во время ноты нажать PC15	Режим меняется, звук останавливается, настройки сохраняются
Клавиатура временно не отвечает по I2C	Ошибочный кадр отбрасывается. Main продолжает обслуживать PC15 и время.

Задача: какую кнопку нашли?

Выбрана строка P2. Столбцы P4, P5, P6 прочитались как 1, 0, 1. Нулём стал P5, то есть второй столбец. Это SB7: row=2, column=1, код=2+4×1+1=7.

В PIANO SB7 даст си. В TEST появится Key 7. Один и тот же распознанный номер получает разный смысл в зависимости от режима.

Задача на фантомную кнопку

Нажаты SB1, SB2 и SB5. В матрице без диодов может дополнительно выглядеть нажатой SB6. Программа отвергнет весь многокнопочный кадр. Она не обещает определить и сыграть все настоящие кнопки из этой комбинации.

Практика 2. Адреса, регистры и время

Что отправить для выбора строки P0?

```
START -> 0xE2 -> 0x03 -> 0xFE -> STOP
```

0xE2 - адрес DD12 с направлением записи. 0x03 - Configuration. 0xFE делает P0 выходом. Output заранее равен нулю, поэтому строка выдаёт ноль. После байтов ожидаются подтверждения ACK.

Что отправить для чтения столбцов?

```
START -> 0xE2 -> 0x00 -> repeated START
      -> 0xE3 -> байт входов -> NACK -> STOP
```

0x00 выбирает Input. После смены направления получаем байт, в котором смотрим P4-P6. NACK в конце здесь нормален: принимаем только один байт.

Проверка чисел

Число	Что означает
0x71	Семибитный адрес DD12 в вызове API
0xE2 / 0xE3	Адресный байт записи / чтения на линии
0x0FFF	Маска 12 позиций кнопок
30 мс	Подтверждение нажатия или полного отпускания
10 мс / 30 мс / 50 мс	Тайм-аут обмена / предел восстановления / пауза до повтора

Пример подтверждения

Полные кадры начали показывать только SB3 в 100 мс. В 129 мс событие ещё рано. В первом следующем подходящем кадре на 130 мс или позже событие может выдаться. Если между ними пришёл кадр с двумя кнопками или ошибкой, потребуется новое полное отпускание.

Что показать и повторить перед защитой

Короткий маршрут демонстрации

1. TEST. После сброса отпустить матрицу. Проверить все 12 кодов и одно сообщение при удержании.

2. Несколько кнопок. Нажать две, отпустить одну: событий быть не должно. Полностью отпустить, затем показать новое одиночное нажатие.

3. PIANO. Нажать PC15, снова отпустить матрицу. Проверить SB1-SB7, октаву SB8/SB9, время SB10/SB11 и гамму SB12.

4. Переход режима. Во время звука нажать PC15. Звук должен закончиться. Проверить, что настройки сохранились и зажатая матричная кнопка не сработала в новом режиме.

5. Регистры и линии. Показать адрес 0x71, значения FE/FD/FB/F7, CCR=188 и TRISE=31. Если есть анализатор - SCL/SDA и repeated START.

6. Музыка. В первой октаве нажать SB6. Показать 440 Гц и настройки TIM2. Для слышимого сигнала нужен замкнутый SA2.7.

Пять фраз для повторения

Матрица даёт строку и столбец. PCA9538 связывает их с I2C. Подтверждается полный кадр. После комбинации нужно отпустить все кнопки. Ошибка обмена не должна останавливать основной цикл.

Сборка и прошивка

```
python3 lab4/tools/build.py
python3 lab4/tools/board.py flash
```

Команды выполняются из корня проекта. Результат - `lab4/build/lab4.elf` и `.bin`. В CubeIDE импортируется отдельный проект lab4, для отладки используется SDK11M.cfg и FT2232/JTAG.

Проверять интервалы нужно без остановок отладчика. Компьютерный ввод в терминал музыкальными командами этой версии не является.

Карта проекта, источники и проверка

Что искать	Файл внутри lab4
Пины, частоты и PD7	Core/Inc/board.h
Обмен и восстановление шины	Core/Src/i2c_bus.c
Матрица, кадр, дребезг и повтор	Core/Src/keypad.c
Боковая кнопка	Core/Src/button.c
Режимы и очередь отчётов	Core/Src/app.c
Музыка и таймер	Core/Src/piano.c, sound.c
Схема матрицы / буфера	docs/SDK1.1M.pdf, страницы PDF 6 / 9
Проверки	tests/test_lab4.c, tests/run_tests.py

Что проверено при подготовке

03.11.2026 прошивка и тестовая сборка собраны в отдельной временной копии проекта. Прошли **182 проверки C-кода и 193 проверки I2C, клавиатуры, приложения и запуска**. Проверен и запуск готовой прошивки от Reset_Handler.

Модель воспроизводит обмен, матрицу и фантомные соединения, но не реальные фронты, USB, усилитель и звук. Физическая работа стенда в рамках подготовки не подтверждалась.

Источники

Основой служат код lab4, README.md, docs/DEFENSE.md, docs/VALIDATION.md и электрическая схема. README относит работу к заданию 1 из редакции пособия 23.09.2026 с пометкой об уточнении задания. Здесь объясняется фактическая реализация проекта.

NXP PCA9538: регистры и протокол расширителя

ST ES0206: ограничения I2C, разделы 2.10.1 и 2.10.4

ST RM0090: контроллер I2C STM32F4